Chaos Experiment - Instance Start and Stop Hands-on

Objective:

Resiliency against an AZ failure

- Launch **2 EC2 instances** in different Availability Zones (`us-east-2a` and `us-east-2b`).
- Use a **Load Balancer** in front of them.
- Simulate **chaos** by stopping the instance in one AZ.
- Show that traffic automatically shifts to the healthy instance in the other AZ.

This simulates **resiliency against an AZ failure**.

Step-by-Step Implementation

1. Terraform Setup

We'll create:

- **VPC + Subnets** (one in each AZ).
- **Security Group** (allow SSH + HTTP).
- **2 ec2 instances** (one in `us-east-2a`, another in `us-east-2b`).
- **Application Load Balancer (ALB)** that distributes traffic.

2. Terraform Code

Here's a **fully automated Terraform script**:
create main.tf file and add below code

……..

```
provider "aws" {
  region = "us-east-2"
}
```

```
# Security Group for EC2 + ALB
resource "aws_security_group" "demo_sg" {
  name        = "chaos-demo-sg"
  description = "Allow SSH and HTTP"
  vpc_id      = data.aws_vpc.default.id

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
```

```
  }
}

# Get default VPC
data "aws_vpc" "default" {
  default = true
}

# Get default subnets
data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

# Instance A in us-east-2a
resource "aws_instance" "instance_a" {
  ami           = "ami-0cfde0ea8edd312d4"
  instance_type = "t2.micro"
  subnet_id     = element(data.aws_subnets.default.ids, 0)
  availability_zone = "us-east-2a"
  vpc_security_group_ids = [aws_security_group.demo_sg.id]
  user_data = <<-EOF
              #!/bin/bash
              sudo apt update -y
```

```
            sudo apt install -y nginx
            echo "Hello from Instance A (us-east-2a)" > /var/www/html/index.html
            sudo systemctl start nginx
            sudo systemctl enable nginxd
            EOF
  tags = {
    Name = "Instance-A"
  }
}

# Instance B in us-east-2b
resource "aws_instance" "instance_b" {
  ami           = "ami-0cfde0ea8edd312d4"
  instance_type = "t2.micro"
  subnet_id     = element(data.aws_subnets.default.ids, 1)
  availability_zone = "us-east-2b"
  vpc_security_group_ids = [aws_security_group.demo_sg.id]
  user_data = <<-EOF
            #!/bin/bash
            sudo apt update -y
            sudo apt install -y nginx
            echo "Hello from Instance B (us-east-2b)" > /var/www/html/index.html
            sudo systemctl enable nginx
            sudo systemctl start nginx
            EOF
  tags = {
```

```
    Name = "Instance-B"
  }
}

# Create Load Balancer
resource "aws_lb" "app_lb" {
  name               = "chaos-demo-lb"
  internal           = false
  load_balancer_type = "application"
  security_groups    = [aws_security_group.demo_sg.id]
  subnets            = data.aws_subnets.default.ids
}

resource "aws_lb_target_group" "app_tg" {
  name     = "chaos-demo-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = data.aws_vpc.default.id
  health_check {
    path = "/"
    port = "80"
  }
}

resource "aws_lb_listener" "app_listener" {
  load_balancer_arn = aws_lb.app_lb.arn
```

```
  port              = "80"
  protocol          = "HTTP"

  default_action {
    type            = "forward"
    target_group_arn = aws_lb_target_group.app_tg.arn
  }
}

# Register instances in Target Group
resource "aws_lb_target_group_attachment" "a" {
  target_group_arn = aws_lb_target_group.app_tg.arn
  target_id        = aws_instance.instance_a.id
  port             = 80
}

resource "aws_lb_target_group_attachment" "b" {
  target_group_arn = aws_lb_target_group.app_tg.arn
  target_id        = aws_instance.instance_b.id
  port             = 80
}

# IAM Role for FIS
resource "aws_iam_role" "fis_role" {
  name = "ChaosFISRole"
```

```
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect    = "Allow"
      Principal = {
        Service = "fis.amazonaws.com"
      }
      Action = "sts:AssumeRole"
    }]
  })
}

# Attach Policy to FIS Role
resource "aws_iam_role_policy" "fis_policy" {
  name = "ChaosFISPolicy"
  role = aws_iam_role.fis_role.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect   = "Allow"
      Action   = [
        "ec2:StopInstances",
        "ec2:StartInstances",
        "ec2:DescribeInstances"
      ]
```

```
      Resource = "*"
    }]
  })
}

# FIS Experiment to stop Instance A
resource "aws_fis_experiment_template" "chaos_experiment" {
  description = "Chaos Engineering Demo: Stop Instance A"
  role_arn    = aws_iam_role.fis_role.arn

  stop_condition {
    source = "none"
  }

  target {
    name   = "TargetInstances"
    resource_type = "aws:ec2:instance"
    selection_mode = "ALL"
    resource_arns = [aws_instance.instance_a.arn]
  }

  action {
    name = "stop-instances"
    action_id = "aws:ec2:stop-instances"

    target {
```

```
    key   = "Instances"
    value = "TargetInstances"
   }
  }

  tags = {
   Name = "ChaosExperiment"
  }
}

output "load_balancer_dns" {
  value = aws_lb.app_lb.dns_name}
```

……

terraform init

terraform plan

terraform apply

(You will see output which is your load balance dns)

…….

Expected Chaos Demo Flow

1. Visit ALB URL → Served by **Instance B**.
2. Run FIS Experiment → Instance B stops.
3. Refresh browser → ALB automatically shifts traffic to **Instance A**.

……

Navigate to AWS FIS (Console)

1. **Login** to your AWS Management Console.
2. In the **top search bar**, type **"Fault Injection Simulator"** or **"FIS"**.
3. Click on **Fault Injection Simulator** from the search results.
4. You'll now land on the **AWS FIS Dashboard**.
    - Left panel options:
        - **Experiment templates** → Pre-defined chaos experiments.
        - **Experiments** → Run and monitor chaos experiments.
        - **Actions** → Start

……..

Steps to Check in Browser

1. **Open your Load Balancer DNS URL** (example: `http://<ALB-DNS>`).
    - You'll see Instance A (Hello).
2. **Start the FIS Experiment (10 min)**
    - Instance A will be stopped.
3. **Refresh the Browser**
    - Now you'll see Instance B page (Hello).

……..

Chaos Handons Summary

1. **Objective**
    - Demonstrate **resilience of infrastructure** by running a controlled failure experiment.

- Show how traffic seamlessly shifts between instances when one fails.

2. **Setup**
   - Two EC2 instances (`Instance A` in `us-east-2a`, `Instance B` in `us-east-2b`).
   - Both registered under an **Application Load Balancer (ALB)**.
   - Only **one instance runs at a time** to serve traffic.
3. **Experiment**
   - Used **AWS Fault Injection Simulator (FIS)** to **stop Instance** (running one).
   - Observed:
     - Instance B stopped.
     - ALB automatically routed traffic to **Instance A**, which started and served requests.
   - After FIS completion, system remained available without downtime.
4. **Outcome**

Demonstrated **high availability** – application stayed online even when one instance failed.
Showed **automatic traffic failover** using ALB health checks.Verified **chaos experiment success** – proved system can tolerate instance failure.

Key Learning

- Chaos Engineering is not about breaking things, but about **building confidence** that the system can recover gracefully.
- With automation (FIS + ALB + EC2), the system becomes **self-healing and resilient**.